

Plan: poner G-NEEX en línea (Camino A)

Documento de referencia personal — Phoenix Cell G-NEEX

Enfoque: publicar la aplicación en Internet **manteniendo el modelo actual** (SPA estática + datos en el navegador / `localStorage`, sincronización vía respaldo JSON).

Fecha de elaboración: abril de 2026

1. Qué es el «Camino A»

- La app sigue siendo **archivos estáticos** (HTML, CSS, JavaScript) servidos por **HTTPS**.
 - **Cada dispositivo** conserva su copia de datos en el almacenamiento local del navegador, igual que hoy.
 - La **fuentes de verdad entre equipos** sigue siendo el flujo que ya tenéis: **exportar / importar respaldo JSON** (y, si aplica, exportar solo movimientos).
 - **No** implica, por sí solo, servidor de base de datos ni edición concurrente multiusuario sobre una única copia en tiempo real.
-

2. Objetivos concretos

1. Tener una **URL pública o accesible** (por ejemplo `https://gneex.ejemplo.com`).
 2. Servir la aplicación con **HTTPS** (certificado TLS válido).
 3. Definir un **procedimiento de publicación** (cómo subir una nueva versión sin romper a quien ya tiene la página abierta).
 4. Dejar por escrito el **ritual de copias** (quién exporta respaldo, cuándo, dónde se guarda el archivo).
-

3. Pasos recomendados (orden lógico)

3.1 Dominio y DNS

- Registrar o usar un **dominio**.
- Crear un registro **DNS** (tipo `A` o `CNAME`) que apunte al proveedor de hosting que elijas.

3.2 Hosting estático

Opciones habituales (elegir una según coste y familiaridad):

- **Cloudflare Pages, Netlify, GitHub Pages**
- **Azure Static Web Apps, AWS S3 + CloudFront**
- Un **VPS** con **Nginx** sirviendo la carpeta del proyecto

Requisitos mínimos: servir archivos estáticos y permitir **HTTPS**.

3.3 HTTPS

- Activar **TLS** en el proveedor (muchas opciones incluyen certificado **Let's Encrypt** automático).
- Forzar redirección **HTTP** → **HTTPS** si el proveedor lo permite.

3.4 Despliegue (publicar una nueva versión)

- **Manual al inicio:** subir ZIP o sincronizar carpeta (`git pull` + copiar `index.html`, `js/`, `css/`, etc.).
- **Más adelante:** repositorio **Git** + acción automática que despliegue al hacer `push` o al crear un **tag** de versión.

3.5 Caché del navegador

- Configurar **caché larga** para `*.js` y `*.css` con **nombres versionados** o parámetro de versión, o caché corta al principio.
- Evitar que los usuarios queden con **HTML nuevo** y **JS antiguo** mezclados: política clara para `index.html` (poca o nula caché).

3.6 Datos y privacidad

- Recordar: **los datos del inventario en producción siguen en cada PC** salvo que importéis un respaldo en otro sitio.
- Si la URL es **pública en Internet**, valorar **no exponer datos sensibles** en el propio código (claves, listas internas); el riesgo principal sigue siendo **quién tiene acceso a la URL** y a las **cuentas** de la app.
- Si hay **datos personales** (usuarios, nombres, auditoría), alinear con política interna o **RGPD** según corresponda.

3.7 Comunicación al equipo

- Una página corta (aunque sea interna): «**Cómo abrimos G-NEEX**», «**cómo exporto respaldo**», «**cómo importo en otro ordenador**», «**qué hago si la página sale en blanco**» (caché / otra versión).

3.8 Versión visible (opcional pero útil)

- Mostrar en la app un **número de versión** o **fecha de build** para saber qué copia está cada uno usando tras un despliegue.

4. Límites que debes conocer (Camino A)

Tema	Límite
Varios editores	No hay una única base en servidor; dos personas pueden desalinear datos si no siguen el ritual de respaldo/import.
Tamaño de datos	<code>localStorage</code> tiene techo aproximado (orden de varios MB según navegador); un inventario enorme puede acercarse al límite.

Tema	Límite
Copia de seguridad	Sigue siendo responsabilidad operativa (exportar JSON con frecuencia y guardarlo fuera del PC).

5. Checklist rápida antes de dar por «en línea» cerrado

- URL con **HTTPS** funcionando.
 - Login y flujos principales probados **desde fuera** de la red local (si aplica).
 - **Exportar respaldo** probado tras el despliegue (misma versión que en local).
 - **Importar respaldo** probado en un navegador limpio o segundo equipo (recarga esperada).
 - Texto o enlace interno con **quién contactar** si falla el hosting o el dominio.
-

6. Si más adelante necesitáis «una sola base para todos»

Eso ya no es Camino A: sería **backend + API + base de datos** (migración desde el JSON de respaldo como punto de partida). Este PDF no sustituye ese diseño; solo delimita hasta dónde llega A.

Fin del documento.